

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 – : Big Data Analytics

Practical File – ODD Semester 2026-27

StudentID: 24DIT066

Student Name: Sil Shah

Practical 4

Aim: To implement a MapReduce program using Python and Hadoop to find the maximum temperature recorded for each year from the NCDC weather dataset.

Problem definition:

Find the Maximum Temperature Recorded for Each Year of NCDC Data using MapReduce
A meteorological research organization maintains historical weather records collected from various weather stations worldwide. The organization uses Hadoop to analyze large climate datasets and study long-term temperature trends. As a Data Analytics Engineer, you are required to process the National Climatic Data Center (NCDC) dataset and identify the maximum temperature recorded for each year using MapReduce.

Task 1: Upload the NCDC weather dataset to HDFS and examine the structure of the weather records. Task 2: Develop and implement the Mapper program to extract the year and temperature values from each record and generate intermediate key-value pairs. Task 3: Develop and implement the Reducer program to determine the maximum temperature recorded for each year. Task 4: Execute the MapReduce job on Hadoop and store the results in HDFS. Verify the correctness of the output generated. Task 5: Analyze the yearly maximum temperature trends and identify the hottest year in the dataset. Discuss the advantages of using MapReduce for large-scale climate data analysis.

Objectives:

To implement a MapReduce program using Python and Hadoop to find the maximum temperature recorded for each year from the NCDC weather dataset and store the results in HDFS.

Tools Used

- Hadoop 3.x
- HDFS
- Python 3
- Hadoop Streaming
- Linux/Kali Ubuntu
- NCDC Weather Dataset

Observation / Output

STEP 1 — CHECK JAVA AND HADOOP

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ java -version
openjdk version "11.0.31" 2026-04-21
OpenJDK Runtime Environment (build 11.0.31+11-post-1ubuntu1-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.31+11-post-1ubuntu1-26.04.2-Ubuntu, mixed mode, sharing)
hadoop@sil-shah-VMware-Virtual-Platform:~$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
```

Step 2: jps output

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ jps
6005 SecondaryNameNode
5749 DataNode
5610 NameNode
6221 ResourceManager
6366 NodeManager
6735 Jps
```

Step 3: CREATE PROJECT DIRECTORY

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ mkdir -p ~/Practical4
hadoop@sil-shah-VMware-Virtual-Platform:~$ cd ~/Practical4
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ pwd
/home/hadoop/Practical4
```

Step 4: CREATE NCDC DATASET

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ mkdir -p ~/Practical4
hadoop@sil-shah-VMware-Virtual-Platform:~$ cd ~/Practical4
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ pwd
/home/hadoop/Practical4
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ nano ncdc_data.txt
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ cat ncdc_data.txt
1990,35
1990,42
1990,38
1990,45
1991,40
1991,43
1991,39
1991,47
1992,41
1992,48
1992,44
1993,39
1993,46
1993,43
1994,50
1994,45
1994,48
```

Step 5: UPLOAD DATASET TO HDFS

```
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ hdfs dfs -mkdir -p /user/$USER/ncdc/input
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ hdfs dfs -put ncdc_data.txt /user/$USER/ncdc/input/
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ hdfs dfs -ls /user/$USER/ncdc/input
Found 1 items
-rw-r--r--  1 hadoop supergroup      136 2026-08-14 14:57 /user/hadoop/ncdc/input/ncdc_data.txt
```

Step 6: CREATE MAPPER

```
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ nano mapper.py
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ chmod +x mapper.py
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ ls -l mapper.py
-rwxrwxr-x 1 hadoop hadoop 325 Aug 14 14:58 mapper.py
```

Step 7: TEST MAPPER

```
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ cat ncdc_data.txt | python3 mapper.py
1990      35
1990      42
1990      38
1990      45
1991      40
1991      43
1991      39
1991      47
1992      41
1992      48
1992      44
1993      39
1993      46
1993      43
1994      50
1994      45
1994      48
```

Step 8: CREATE REDUCER

```
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ nano reducer.py
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ chmod +x reducer.py
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ ls -l reducer.py
-rwxrwxr-x 1 hadoop hadoop 722 Aug 14 15:09 reducer.py
```

Step 9: TEST MAPPER + REDUCER

```
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ cat ncdc_data.txt | python3 mapper.py | sort | python3 reducer.py
1990      45
1991      47
1992      48
1993      46
1994      50
```

Step 10: FIND HADOOP STREAMING JAR

```
hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ find $HADOOP_HOME -name "hadoop-streaming*.jar"
/usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar
/usr/local/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.3.6-sources.jar
/usr/local/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.3.6-test-sources.jar
```

Step 11: REMOVE OLD OUTPUT & EXECUTE MAPREDUCE JOB

```

Total Committed heap usage (bytes)=459276288
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=136
File Output Format Counters
  Bytes Written=40
2026-08-14 15:18:06,398 INFO streaming.StreamJob: Output directory: /user/hadoop/ncdc/output

```

Step 12: CHECK HDFS OUTPUT

```

hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ hdfs dfs -ls /user/$USER/ncdc/output
Found 2 items
-rw-r--r--  1 hadoop supergroup          0 2026-08-14 15:18 /user/hadoop/ncdc/output/_SUCCESS
-rw-r--r--  1 hadoop supergroup       40 2026-08-14 15:18 /user/hadoop/ncdc/output/part-00000

```

Step 13: DISPLAY FINAL RESULT

```

hadoop@sil-shah-VMware-Virtual-Platform:~/Practical4$ hdfs dfs -cat /user/$USER/ncdc/output/part-00000
1990    45
1991    47
1992    48
1993    46
1994    50

```

Year	Maximum Temperature
1990	45
1991	47
1992	48
1993	46
1994	50

Result:

The MapReduce program was successfully implemented using Python and Hadoop Streaming. The maximum temperature for each year was calculated and the results were successfully stored and retrieved from HDFS. The hottest year in the given dataset was 1994, with a maximum temperature of 50.

Key Questions / Analysis

Q1. How is weather data parsed and processed using MapReduce?

The Mapper reads each weather record, extracts the year and temperature, and generates key-value pairs. Hadoop then groups the data by year, and the Reducer finds the maximum temperature for each year.

Q2. What key-value pairs are generated by the Mapper?

The Mapper generates **(Year, Temperature)** pairs. For example, 1990,45 becomes 1990 45.

Q3. How does the Reducer identify the maximum temperature for a given year?

The Reducer compares all temperature values belonging to the same year and selects the highest value.

Q4. What challenges arise when processing large climate datasets?

Large data volume, missing or invalid records, different data formats, storage requirements, and longer processing time are common challenges.

Q5. How does Hadoop improve scalability and performance for weather data analytics?

Hadoop distributes data and processing across multiple nodes, allowing large datasets to be processed efficiently using parallel processing.

Conclusion

The MapReduce program was successfully implemented using Python and Hadoop to find the maximum temperature recorded for each year. The Mapper generated year-temperature pairs, while the Reducer calculated the maximum temperature for each year. The final results were stored and verified in HDFS.